Министерство науки и высшего образования Российской Федерации
ФГАОУ ВО «Уральский федеральный университет
имени первого Президента России Б.Н. Ельцина»
Институт новых материалов и технологий
Кафедра «Теплофизика и информатика в металлургии»

Использование Prometheus и Grafana для эффективного мониторинга ASP.NET Core приложений

ОТЧЕТ
по практической работе № 5
по дисциплине «Основы методологии Development Operation»

Направление 09.03.02 «Информационные системы и технологии» (уровень бакалавриата)
Образовательная программа
09.03.02/33.02 «Информационные системы и технологии» (СУОС)

Студент
группы НМТ-433901 О.М. Малышева

Преподаватель:
профессор, д.т.н. В.В.Лавров

Екатеринбург
2026

СОДЕРЖАНИЕ

1.1 Цель работы 3
1.2 Ход проведения работы 3
1.2.1 Разработка тестового приложения Visual Studio .NET Core 3
1.2.2 Создание файла docker-compose.yml для запуска простой конфигурации Prometeus и Grafana 4
1.2.3 Настройка подключения Grafana к Prometeus и подготовка дашборда 4
1.2.4 Демонстрация мониторинга работы веб-приложения при корректировке 6
1.3 Выводы 6
Приложение А Листинг программного кода приложения «Калькулятор» (контроллер) 8
Приложение Б Листинг программного кода приложения «Калькулятор» (appsettings.json) 9
Приложение В Листинг программного кода приложения «Калькулятор» (appsettings.Development.json) 10
Приложение Г Листинг программного кода приложения «Калькулятор» (Program.cs) 11
Приложение Д Листинг файла docker-compose.yml 13

Практическая работа
«Использование Prometheus и Grafana для эффективного мониторинга ASP.NET Core приложений»

1.1 Цель работы

Цель работы — разработать и развернуть тестовое веб-приложение на платформе Visual Studio .NET Core, настроить сбор его метрик с помощью Prometheus и организовать визуальный мониторинг состояния и производительности приложения в Grafana.

В ходе работы необходимо обеспечить запуск компонентов мониторинга в контейнерах Docker, настроить получение метрик веб-приложения и представить результаты наблюдения в виде графиков и показателей на панели мониторинга.

1.2 Ход проведения работы

1.2.1 Разработка тестового приложения Visual Studio .NET Core

В качестве тестового приложения использовано веб-приложение «Калькулятор», разработанное в Visual Studio на платформе .NET. Для сбора метрик в проект подключались пакеты OpenTelemetry и Prometheus Exporter, обеспечивающие инструментирование приложения и предоставление данных мониторинга через HTTP endpoint.

На рисунке 1 показано окно управления пакетами NuGet проекта, в котором отображается установленный пакет SharpAbp OpenTelemetry Exporter. Подключение компонентов OpenTelemetry необходимо для последующей передачи метрик приложения в систему мониторинга.

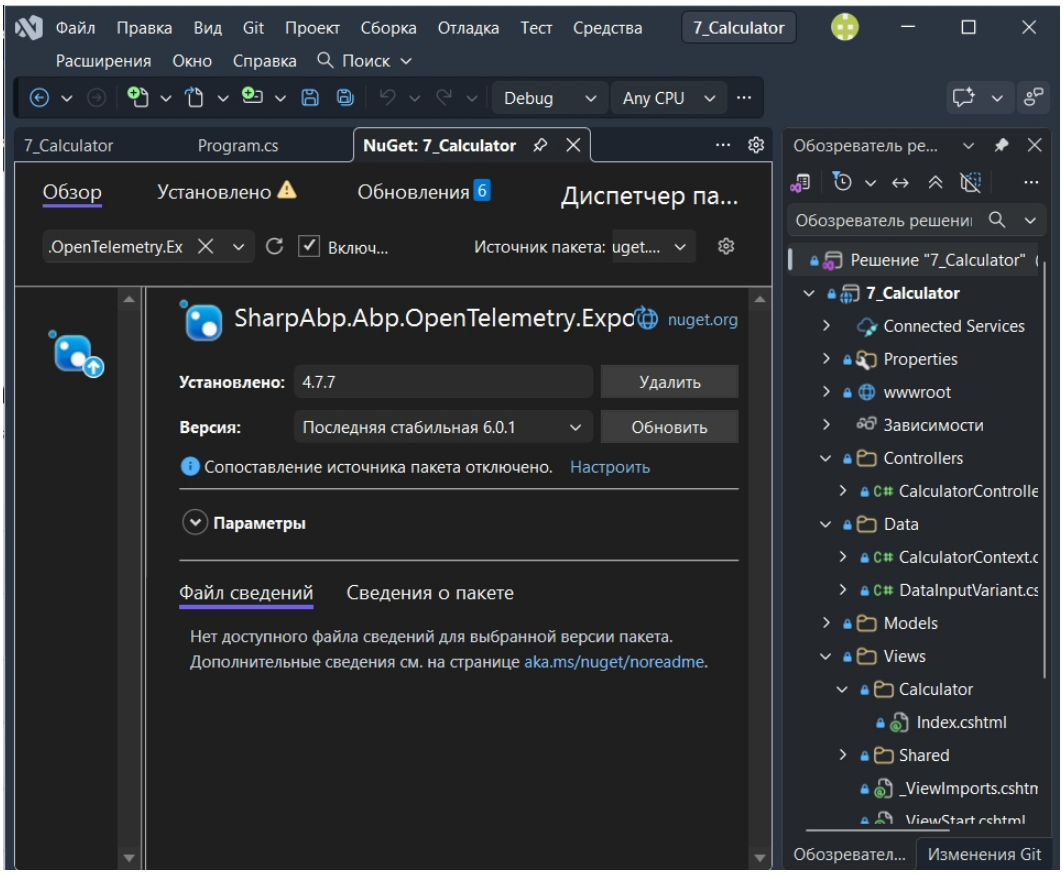


Рисунок 1 — Подключение пакета OpenTelemetry Exporter в проекте Visual Studio

1.2.2 Создание файла docker-compose.yml для запуска простой конфигурации Prometeus и Grafana

Для настройки сбора метрик был подготовлен конфигурационный файл Prometheus. В нём задан интервал опроса 10 секунд, имя задания olesya-calculator, путь получения метрик /metrics и адрес тестового приложения host.docker.internal:5007. Такая конфигурация определяет, откуда Prometheus получает данные о работе веб-приложения.

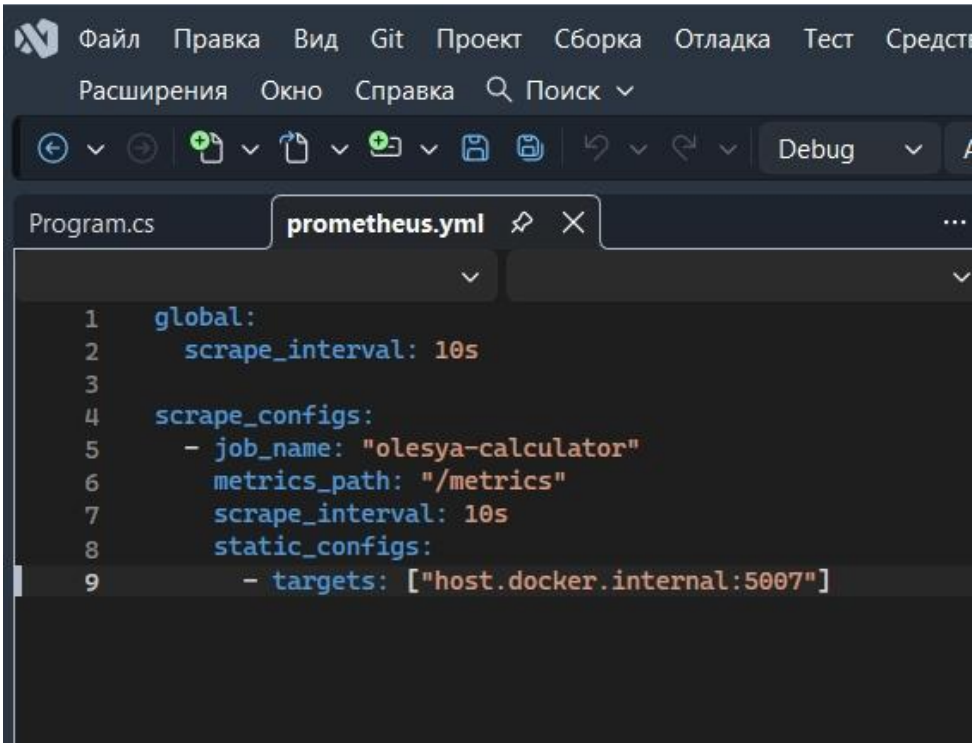


Рисунок 2 — Конфигурация Prometheus в файле prometheus.yml

Запуск Prometheus и Grafana выполнялся с помощью Docker Compose. После выполнения команды docker compose up -d были запущены контейнеры olesya-prometheus и olesya-grafana. Дополнительно командой docker ps проверено состояние контейнеров и подтверждено, что контейнер калькулятора также продолжает работать и доступен на порту 5007.

```
student@sk5-cod-tim-student:~/433901_devops_MalyshevaOlesya/App_practical/7_Calculator/Calcul
ator$ sudo docker compose up -d
WARN[0000] Found orphan containers ([web-mariadb-1 web-app-calculator-liliya calculator-rende
rer-1]) for this project. If you removed or renamed this service in your compose file, you ca
n run this command with the --remove-orphans flag to clean it up.
[+] Running 2/2
 ✔ Container olesya-grafana     Started                                                    0.5s
 ✔ Container olesya-prometheus  Started                                                    0.5s
student@sk5-cod-tim-student:~/433901_devops_MalyshevaOlesya/App_practical/7_Calculator/Calcul
ator$ sudo docker ps --filter "name=calculator_olesya_5007" --filter "name=olesya-prometheus"
 --filter "name=olesya-grafana"
CONTAINER ID   IMAGE                    COMMAND                  CREATED          STATUS
      PORTS                                       NAMES
ccd520d80141   grafana/grafana:latest   "/run.sh"                32 seconds ago   Up 31 seco
nds                                               olesya-grafana
11ad17461f6d   prom/prometheus:v2.54.1  "/bin/prometheus --c…"   32 seconds ago   Up 31 seco
nds                                               olesya-prometheus
0697711d185b   calculator:latest        "dotnet 7_Calculator…"   42 minutes ago   Up 42 minu
tes   0.0.0.0:5007->5007/tcp, :::5007->5007/tcp   calculator_olesya_5007
```

Рисунок 3 — Запуск контейнеров Prometheus и Grafana средствами Docker Compose

1.2.3 Настройка подключения Grafana к Prometeus и подготовка дашборда

После запуска служб была проверена доступность целевого endpoint Prometheus. На странице Targets отображается задание olesya-calculator со статусом UP. Это означает, что Prometheus успешно обращается к адресу приложения и выполняет опрос указанного endpoint /metrics.

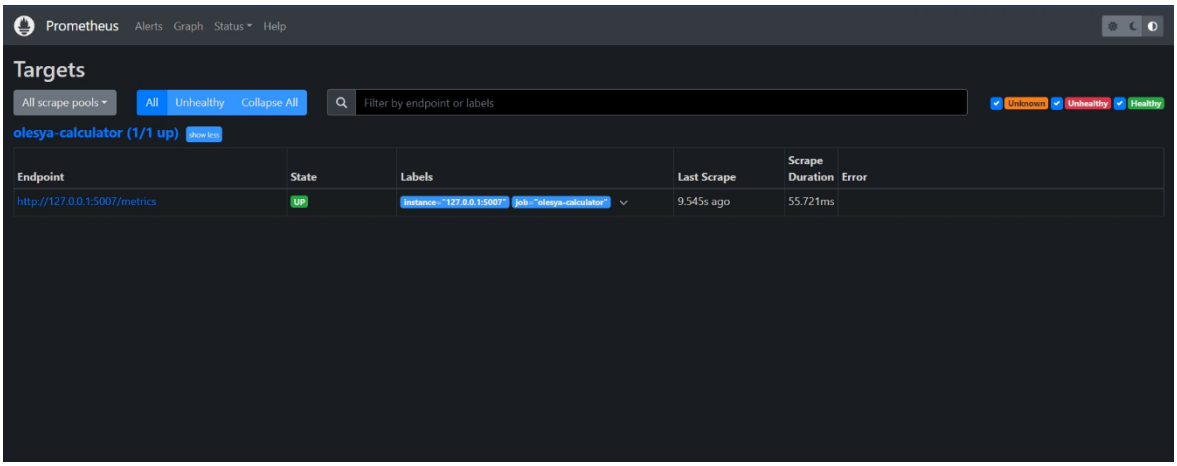


Рисунок 4 — Проверка состояния цели мониторинга в Prometheus

Для анализа собранных данных в Prometheus была выполнена проверка метрики http_server_request_duration_seconds_count. Графическое представление показывает изменение количества зарегистрированных HTTP-запросов во времени и позволяет убедиться в наличии данных для мониторинга.

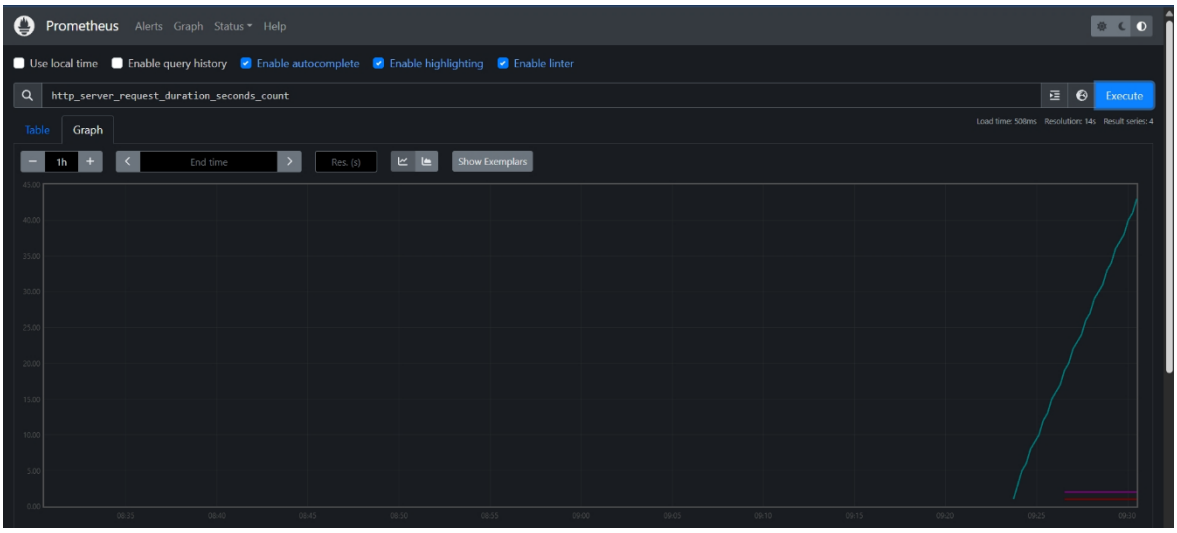


Рисунок 5 — График метрики количества HTTP-запросов в Prometheus

В Grafana был добавлен источник данных Prometheus. В качестве типа источника выбран Prometheus, после чего настроено подключение к серверу мониторинга. Успешная настройка источника позволяет использовать собранные Prometheus временные ряды при создании панелей Grafana.

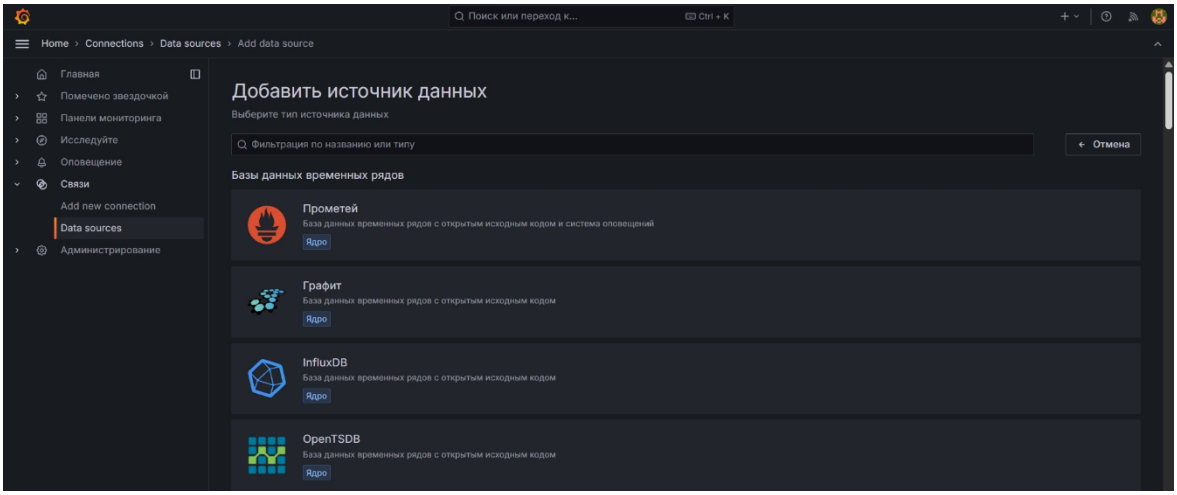


Рисунок 6 — Выбор источника данных Prometheus в Grafana

1.2.4 Демонстрация мониторинга работы веб-приложения при корректировке

На заключительном этапе был подготовлен дашборд Grafana для наблюдения за основными показателями работы веб-приложения. На панели представлены количество подключений Kestrel в очереди, число активных подключений Kestrel, количество активных HTTP-запросов, среднее время обработки HTTP-запроса и частота поступления HTTP-запросов.

При обращении к веб-приложению и выполнении запросов значения показателей и графики изменяются во времени. Это позволяет наблюдать реакцию приложения на поступающую нагрузку, оценивать активность HTTP-взаимодействий и контролировать время обработки запросов.

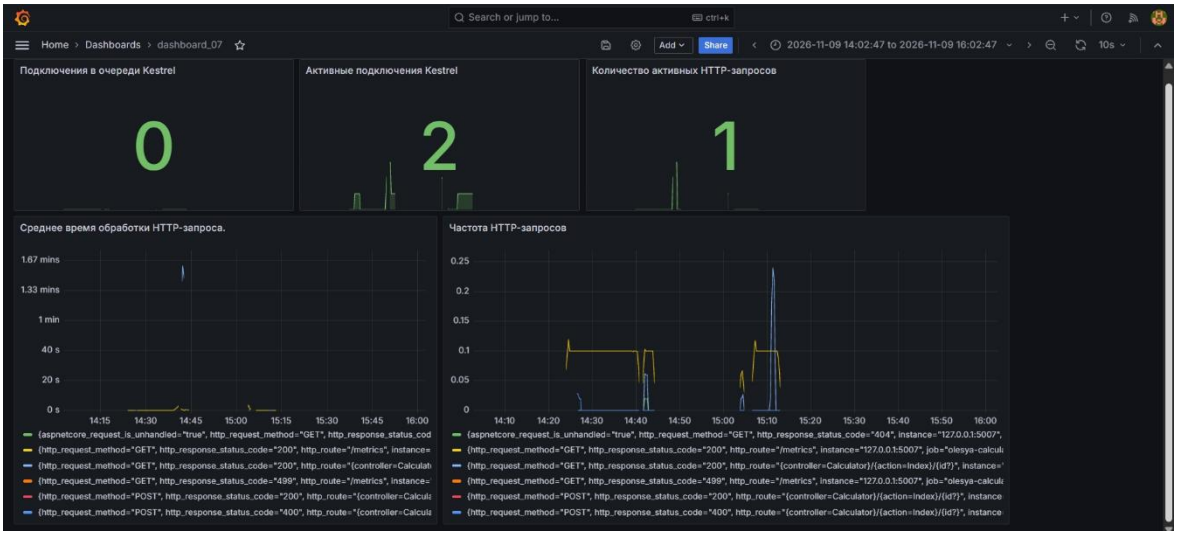


Рисунок 7 — Итоговый дашборд мониторинга веб-приложения «Калькулятор» в Grafana

1.3 Выводы

В ходе лабораторной работы было подготовлено тестовое веб-приложение «Калькулятор» на платформе Visual Studio .NET Core и настроен сбор метрик с использованием OpenTelemetry и Prometheus. С помощью Docker Compose были запущены контейнеры Prometheus и Grafana. В Prometheus проверена доступность endpoint приложения и получение метрик, а в Grafana настроен источник данных Prometheus и создан дашборд с показателями HTTP-запросов и подключений Kestrel.

В результате работы реализована базовая система мониторинга веб-приложения. Полученные графики и числовые показатели позволяют отслеживать активность приложения и оценивать его работу при поступлении запросов.

Приложение А Листинг программного кода приложения «Калькулятор» (CalculatorController.cs)

```
using Microsoft.AspNetCore.Mvc;

namespace Calculator.Controllers
{
    public enum Operation
    {
        Add,
        Subtract,
        Multiply,
        Divide
    }

    public class CalculatorController : Controller
    {
        [HttpGet]
        public IActionResult Index()
        {
            return View();
        }

        [HttpPost]
        [ValidateAntiForgeryToken]
        public IActionResult Calculate(double num1, double num2, Operation operation)
        {
            double result = 0;

            switch (operation)
            {
                case Operation.Add:
                    result = num1 + num2;
                    break;

                case Operation.Subtract:
                    result = num1 - num2;
                    break;

                case Operation.Multiply:
                    result = num1 * num2;
                    break;

                case Operation.Divide:
                    if (num2 == 0)
                    {
                        ModelState.AddModelError("", "На ноль делить нельзя.");
                        return View("Index");
                    }

                    result = num1 / num2;
                    break;
            }

            ViewBag.Result = result;

            return View("Index");
        }
    }
}
```

Приложение Б Листинг программного кода приложения «Калькулятор» (appsettings.json)

```
{
  "ConnectionStrings": {
    "DefaultConnection": "Server=mariadb; Database=myWebCalculatorDb; Uid=root;
Pwd=<пароль>; Character Set=utf8; ConvertZeroDatetime=True;"
  },
  "Kestrel": {
    "Endpoints": {
      "Http": {
        "Url": "http://0.0.0.0:5007"
      }
    }
  },
  "Logging": {
    "LogLevel": {
      "Default": "Information",
      "Microsoft": "Warning",
      "Microsoft.Hosting.Lifetime": "Information"
    }
  },
  "AllowedHosts": "*"
}
```

Приложение В Листинг программного кода приложения «Калькулятор» (appsettings.Development.json)

```
{
  "ConnectionStrings": {
    "DefaultConnection": "Server=93.88.178.186; Port=5023; Database=myWebCalcula-
torDb; Uid=root; Pwd=<пароль>; Character Set=utf8; ConvertZeroDatetime=True;"
  },
  "DetailedErrors": true,
  "Logging": {
    "LogLevel": {
      "Default": "Information",
      "Microsoft": "Warning",
      "Microsoft.Hosting.Lifetime": "Information"
    }
  }
}
```

Приложение Г Листинг программного кода приложения «Калькулятор» (Program.cs)

```
using Microsoft.EntityFrameworkCore;
using Calculator.Data;
using OpenTelemetry.Metrics;

var builder = WebApplication.CreateBuilder(args);

// Добавление сервисов
builder.Services.AddControllersWithViews();

string mariadbCS = builder.Configuration.GetConnectionString("DefaultConnection");

builder.Services.AddDbContext<CalculatorContext>(options =>
{
    options.UseMySql(
        mariadbCS,
        new MySqlServerVersion(new Version(10, 5, 15))
    );
});

builder.Services.AddRazorPages();

// Настройка OpenTelemetry и метрик Prometheus
builder.Services.AddOpenTelemetry()
    .WithMetrics(metrics =>
    {
        metrics.AddPrometheusExporter();

        metrics.AddMeter(
            "Microsoft.AspNetCore.Hosting",
            "Microsoft.AspNetCore.Server.Kestrel",
            "Microsoft.AspNetCore.Http.Connections"
        );

        metrics.AddView(
            "http.server.request.duration",
            new ExplicitBucketHistogramConfiguration
            {
                Boundaries = new double[]
                {
                    0, 0.005, 0.01, 0.025, 0.05,
                    0.075, 0.1, 0.25, 0.5, 0.75,
                    1, 2.5, 5, 7.5, 10
                }
            }
        );
    });

var app = builder.Build();

// Настройка HTTP-конвейера
if (!app.Environment.IsDevelopment())
{
    app.UseExceptionHandler("/Calculator/Error");
    app.UseHsts();
}

app.UseHttpsRedirection();
app.UseStaticFiles();
```

```
app.UseRouting();

app.UseAuthorization();

// Endpoint для сбора метрик Prometheus
app.MapPrometheusScrapingEndpoint();

app.MapControllerRoute(
    name: "default",
    pattern: "{controller=Calculator}/{action=Index}/{id?}");

app.Run();
```

Приложение Д Листинг файла docker-compose.yml

```
services:
  prometheus:
    image: prom/prometheus:v2.54.1
    container_name: olesya-prometheus
    ports:
      - "9198:9090"
    volumes:
      - ./prometheus.yml:/etc/prometheus/prometheus.yml:ro
      - prometheus_data:/prometheus
    extra_hosts:
      - "host.docker.internal:host-gateway"
    command:
      - "--config.file=/etc/prometheus/prometheus.yml"
    restart: unless-stopped

  grafana:
    image: grafana/grafana:latest
    container_name: olesya-grafana
    ports:
      - "3018:3000"
    volumes:
      - grafana_data:/var/lib/grafana
    depends_on:
      - prometheus
    restart: unless-stopped

volumes:
  prometheus_data:
  grafana_data:
```